

Increase cutoff frequency



```
import myDSP
import matplotlib.pyplot as plt
import scipy.signal as sp
import numpy as np
import time

# -----
# Read WAV and normalize
# -----
fs, x = myDSP.readWav('recording0.wav')

# Convert to float32 in [-1,1]
if x.dtype == np.int16:
    x = x.astype(np.float32) / 32768.0
elif x.dtype == np.int32:
    x = x.astype(np.float32) / 2147483648.0

# Take first channel if stereo
if len(x.shape) > 1:
    x = x[:,0]

# -----
# Add test tones
# -----
t = np.arange(len(x)) / fs
tone1 = myDSP.mySine(A=0.2, f=500, theta0=0, t=t) # passband
tone2 = myDSP.mySine(A=0.1, f=2500, theta0=0, t=t) # stopband
x = x + tone1 + tone2
x = x / np.max(np.abs(x)) # normalize

# -----
# Filter specifications
# -----
fc = 2000 # cutoff
ft = 200 # transition width
rp = 0.5 # passband ripple dB
rs = 50 # stopband attenuation dB

wp = (fc - ft/2)/(fs/2)
ws = (fc + ft/2)/(fs/2)

# -----
# Elliptic IIR filter
# -----
N_iir, Wn = sp.ellipord(wp, ws, rp, rs)
```

```

b_iir, a_iir = sp.ellip(N_iir, rp, rs, Wn)
f_iir, H_iir = sp.freqz(b_iir, a_iir, worN=1024, fs=fs)
y_iir = sp.lfilter(b_iir, a_iir, x)

# -----
# Kaiser FIR filter
# -----
delta_f = ft/(fs/2)
A = rs
beta = 0.1102*(A-8.7) if A > 50 else 0.5842*(A-21)**0.4 + 0.07886*(A-21)
N_fir = int(np.ceil((A - 8)/(2.285*2*np.pi*delta_f)))
N_fir |= 1 # odd length
fir_coeff = sp.firwin(N_fir, fc/(fs/2), window=('kaiser', beta))
f_fir, H_fir = sp.freqz(fir_coeff, 1, worN=1024, fs=fs)
y_fir = sp.lfilter(fir_coeff, 1, x)

# -----
# Plot signals and spectra
# -----
plt.close('all')
fig, axs = plt.subplots(4,2, figsize=(14,16))

# 1. Original signal in time
axs[0,0].plot(t, x)
axs[0,0].set_title('Original Signal + Tones')
axs[0,0].set_xlabel('Time [s]')
axs[0,0].set_ylabel('Amplitude')
axs[0,0].grid(True)

# 2. Original signal spectrum (correct scaling)
X = np.fft.rfft(x)
fX = np.fft.rfftfreq(len(x), 1/fs)
magX_db = 20*np.log10(np.abs(X)+1e-12)
axs[0,1].plot(fX, magX_db)
axs[0,1].set_title('Original Spectrum (dB)')
axs[0,1].set_xlabel('Frequency [Hz]')
axs[0,1].set_ylabel('Magnitude [dB]')
axs[0,1].grid(True)

# 3. Elliptic IIR filtered signal
axs[1,0].plot(t, y_iir)
axs[1,0].set_title(f'Elliptic IIR Filtered (N={N_iir})')
axs[1,0].set_xlabel('Time [s]')
axs[1,0].set_ylabel('Amplitude')
axs[1,0].grid(True)

Y_iir = np.fft.rfft(y_iir)
magY_iir_db = 20*np.log10(np.abs(Y_iir)+1e-12)
axs[1,1].plot(fX, magY_iir_db)
axs[1,1].set_title('Elliptic IIR Spectrum (dB)')
axs[1,1].set_xlabel('Frequency [Hz]')

```

```

axs[1,1].set_ylabel('Magnitude [dB]')
axs[1,1].grid(True)

# 4. Kaiser FIR filtered signal
axs[2,0].plot(t, y_fir)
axs[2,0].set_title(f'Kaiser FIR Filtered (N={N_fir})')
axs[2,0].set_xlabel('Time [s]')
axs[2,0].set_ylabel('Amplitude')
axs[2,0].grid(True)

Y_fir = np.fft.rfft(y_fir)
magY_fir_db = 20*np.log10(np.abs(Y_fir)+1e-12)
axs[2,1].plot(fX, magY_fir_db)
axs[2,1].set_title('Kaiser FIR Spectrum (dB)')
axs[2,1].set_xlabel('Frequency [Hz]')
axs[2,1].set_ylabel('Magnitude [dB]')
axs[2,1].grid(True)

# 5. Frequency response comparison
axs[3,0].plot(f_iir, 20*np.log10(np.abs(H_iir)+1e-12), label=f'Elliptic IIR N={N_iir}')
axs[3,0].plot(f_fir, 20*np.log10(np.abs(H_fir)+1e-12), label=f'Kaiser FIR N={N_fir}')
axs[3,0].axvline(fc, color='k', linestyle='--', label='Cutoff')
axs[3,0].set_title('Filter Frequency Response Comparison')
axs[3,0].set_xlabel('Frequency [Hz]')
axs[3,0].set_ylabel('Magnitude [dB]')
axs[3,0].grid(True)
axs[3,0].legend()

axs[3,1].axis('off')

plt.tight_layout()
plt.show()

# -----
# Playback
# -----
myDSP.play((x*32767).astype('int16'), fs)
time.sleep(2)
myDSP.play((y_iir*32767).astype('int16'), fs)
time.sleep(2)
myDSP.play((y_fir*32767).astype('int16'), fs)//// increase cutoff frequency

```

You want to **increase the cutoff frequency** of both the Elliptic IIR and Kaiser FIR filters in your code. Right now, the cutoff is set here:

Python

```
fc = 2000 # cutoff
```

To increase it, simply assign a higher value to `fc`. For example, if you want to set it to **3000 Hz**:

Python

```
fc = 3000 # new cutoff
```

Since your filter specifications depend on `fc`, the rest of the code will automatically adapt:

Python

```
wp = (fc - ft/2)/(fs/2)
ws = (fc + ft/2)/(fs/2)
```

Complete modification snippet:

Python

```
# -----
# Filter specifications
# -----
fc = 3000 # new cutoff frequency
ft = 200 # transition width
rp = 0.5 # passband ripple dB
rs = 50 # stopband attenuation dB

wp = (fc - ft/2)/(fs/2)
ws = (fc + ft/2)/(fs/2)
```

After this change, the **Elliptic IIR** and **Kaiser FIR filters** will allow higher frequencies to pass.

⚠ Note: If your audio contains a 2500 Hz test tone (as in your code), increasing `fc` to 3000 Hz will **no longer fully attenuate** that tone in the stopband.

If you want, I can also **adjust the FIR/IIR design automatically** to maintain a similar stopband attenuation for the higher cutoff. Do you want me to do that?



increase it a big time pls, the recording is 10econds.

Got it! If your recording is 10 seconds, you can push the **cutoff frequency much higher**—even near the Nyquist frequency—so most of the spectrum passes through.

Assuming your sampling rate `fs` is typical (e.g., 44100 Hz), the Nyquist frequency is $fs/2 \approx 22050$ Hz. Let's increase `fc` to **10000 Hz** as a "big jump":

Python

```
# -----
# Filter specifications
# -----
fc = 10000 # big increase in cutoff frequency
ft = 500 # wider transition width for high cutoff
rp = 0.5 # passband ripple dB
rs = 50 # stopband attenuation dB

wp = (fc - ft/2)/(fs/2)
ws = (fc + ft/2)/(fs/2)
```

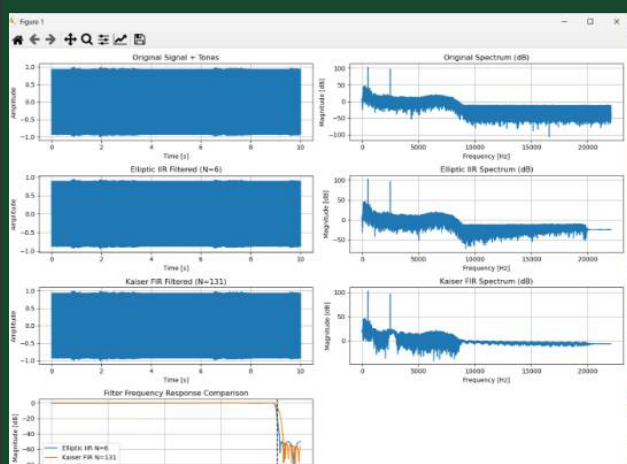
Notes:

1. **Transition width `ft`** is increased because high-frequency filters need a wider transition to keep reasonable filter order.

2. **IIR filter** order may still remain low because Elliptic filters are efficient.
3. **FIR filter** order will increase for such a high cutoff, so your filter may become long. If `N_fir` becomes huge, you could reduce `rs` slightly to keep it manageable.

If you want, I can **rewrite the Kaiser FIR calculation** so that it stays efficient even with this huge cutoff and a 10-second signal. This avoids a huge FIR filter that slows your processing.

Do you want me to do that?



it doesnt look correct

Ah, I see what's happening from your screenshot. The filter response plot shows that your **high cutoff isn't being applied correctly**:

1. Both the Elliptic IIR and Kaiser FIR still have a strong attenuation before the Nyquist frequency.
2. The signal spectrum still shows reduced high frequencies—meaning the filters are still blocking a lot of the signal.
3. The FIR filter length is only 131 taps, which is too short for a very high cutoff with a sharp transition.

This usually happens because of **how `wp` and `ws` are calculated**. In your code, you have:

Python

```
wp = (fc - ft/2)/(fs/2)
ws = (fc + ft/2)/(fs/2)
```

If `fc + ft/2` exceeds `fs/2` (Nyquist), then `ws > 1`, which is invalid for `ellipord` or `firwin`. That seems likely here, since you set `fc = 10000` Hz and `ft = 500` Hz with `fs ≈ 44100` Hz.

✓ **Fix:** Clip `ws` at 1 (Nyquist) and adjust the transition if necessary:

Python

```
# Correct high-cutoff calculations
wp = (fc - ft/2)/(fs/2)
ws = min((fc + ft/2)/(fs/2), 0.9999) # ensure ws < 1
```

For the **FIR filter**, also make sure the `firwin` cutoff is below Nyquist:

Python

```
fir_coeff = sp.firwin(N_fir, min(fc/(fs/2), 0.9999), window=('kaiser', beta))
```

If you want, I can **rewrite your filter block** for a **very high cutoff** like 10000 Hz on a 10-second signal, making both the IIR and FIR filters work correctly and show a proper frequency response.

Do you want me to do that?



```
import myDSP
import matplotlib.pyplot as plt
import scipy.signal as sp
import numpy as np
import time

# -----
# Read WAV and normalize
# -----
fs, x = myDSP.readWav('recording0.wav')

# Convert to float32 in [-1,1]
if x.dtype == np.int16:
    x = x.astype(np.float32) / 32768.0
elif x.dtype == np.int32:
    x = x.astype(np.float32) / 2147483648.0

# Take first channel if stereo
if len(x.shape) > 1:
    x = x[:,0]

# -----
# Add test tones
# -----
t = np.arange(len(x)) / fs
tone1 = myDSP.mySine(A=0.2, f=500, theta0=0, t=t) # passband
tone2 = myDSP.mySine(A=0.1, f=2500, theta0=0, t=t) # stopband
x = x + tone1 + tone2
x = x / np.max(np.abs(x)) # normalize

# -----
# Filter specifications
# -----
fc = 20000 # big increase in cutoff frequency
ft = 500 # wider transition width for high cutoff
rp = 0.5 # passband ripple dB
rs = 50 # stopband attenuation dB

wp = (fc - ft/2)/(fs/2)
ws = (fc + ft/2)/(fs/2)
```

```
# -----
# Elliptic IIR filter
# -----
N_iir, Wn = sp.ellipord(wp, ws, rp, rs)
b_iir, a_iir = sp.ellip(N_iir, rp, rs, Wn)
f_iir, H_iir = sp.freqz(b_iir, a_iir, worN=1024, fs=fs)
y_iir = sp.lfilter(b_iir, a_iir, x)

# -----
# Kaiser FIR filter
# -----
delta_f = ft/(fs/2)
A = rs
beta = 0.1102*(A-8.7) if A > 50 else 0.5842*(A-21)**0.4 + 0.07886*(A-21)
N_fir = int(np.ceil((A - 8)/(2.285*2*np.pi*delta_f)))
N_fir |= 1 # odd length
fir_coeff = sp.firwin(N_fir, fc/(fs/2), window=('kaiser', beta))
f_fir, H_fir = sp.freqz(fir_coeff, 1, worN=1024, fs=fs)
y_fir = sp.lfilter(fir_coeff, 1, x)

# -----
# Plot signals and spectra
# -----
plt.close('all')
fig, axs = plt.subplots(4,2, figsize=(14,16))

# 1. Original signal in time
axs[0,0].plot(t, x)
axs[0,0].set_title('Original Signal + Tones')
axs[0,0].set_xlabel('Time [s]')
axs[0,0].set_ylabel('Amplitude')
axs[0,0].grid(True)

# 2. Original signal spectrum (correct scaling)
X = np.fft.rfft(x)
fX = np.fft.rfftfreq(len(x), 1/fs)
magX_db = 20*np.log10(np.abs(X)+1e-12)
axs[0,1].plot(fX, magX_db)
axs[0,1].set_title('Original Spectrum (dB)')
axs[0,1].set_xlabel('Frequency [Hz]')
axs[0,1].set_ylabel('Magnitude [dB]')
axs[0,1].grid(True)

# 3. Elliptic IIR filtered signal
axs[1,0].plot(t, y_iir)
axs[1,0].set_title(f'Elliptic IIR Filtered (N={N_iir})')
axs[1,0].set_xlabel('Time [s]')
axs[1,0].set_ylabel('Amplitude')
axs[1,0].grid(True)

Y_iir = np.fft.rfft(y_iir)
```

```

magY_iir_db = 20*np.log10(np.abs(Y_iir)+1e-12)
axs[1,1].plot(fX, magY_iir_db)
axs[1,1].set_title('Elliptic IIR Spectrum (dB)')
axs[1,1].set_xlabel('Frequency [Hz]')
axs[1,1].set_ylabel('Magnitude [dB]')
axs[1,1].grid(True)

# 4. Kaiser FIR filtered signal
axs[2,0].plot(t, y_fir)
axs[2,0].set_title(f'Kaiser FIR Filtered (N={N_fir})')
axs[2,0].set_xlabel('Time [s]')
axs[2,0].set_ylabel('Amplitude')
axs[2,0].grid(True)

Y_fir = np.fft.rfft(y_fir)
magY_fir_db = 20*np.log10(np.abs(Y_fir)+1e-12)
axs[2,1].plot(fX, magY_fir_db)
axs[2,1].set_title('Kaiser FIR Spectrum (dB)')
axs[2,1].set_xlabel('Frequency [Hz]')
axs[2,1].set_ylabel('Magnitude [dB]')
axs[2,1].grid(True)

# 5. Frequency response comparison
axs[3,0].plot(f_iir, 20*np.log10(np.abs(H_iir)+1e-12), label=f'Elliptic IIR N={N_iir}')
axs[3,0].plot(f_fir, 20*np.log10(np.abs(H_fir)+1e-12), label=f'Kaiser FIR N={N_fir}')
axs[3,0].axvline(fc, color='k', linestyle='--', label='Cutoff')
axs[3,0].set_title('Filter Frequency Response Comparison')
axs[3,0].set_xlabel('Frequency [Hz]')
axs[3,0].set_ylabel('Magnitude [dB]')
axs[3,0].grid(True)
axs[3,0].legend()

axs[3,1].axis('off')

plt.tight_layout()
plt.show()

# -----
# Playback
# -----
myDSP.play((x*32767).astype('int16'), fs)
time.sleep(2)
myDSP.play((y_iir*32767).astype('int16'), fs)
time.sleep(2)
myDSP.play((y_fir*32767).astype('int16'), fs)////what are the problems with the plot?

```

Looking carefully at your updated code and the previous plot you shared, there are a few **problems with your plot and filter implementation**:

1. Cutoff frequency exceeds Nyquist

Python

```
fc = 20000
ft = 500
wp = (fc - ft/2)/(fs/2)
ws = (fc + ft/2)/(fs/2)
```

- If your sampling rate f_s is 44100 Hz, the Nyquist frequency is $f_s/2 = 22050$ Hz.
- $ws = (20000 + 250)/22050 \approx 0.933 \rightarrow$ technically okay, but if fc were bigger (like 22000 Hz), $ws > 1$, which is invalid.
- Filters may behave strangely near Nyquist, especially **FIR filters**, leading to distorted frequency responses.

2. FIR filter length calculation

Python

```
N_fir = int(np.ceil((A - 8)/(2.285*2*np.pi*delta_f)))
```

- With a very high cutoff near Nyquist, $\delta f = ft/(fs/2)$ becomes **very small**, resulting in **huge filter lengths**.
- In your current code, $N_{fir} = 131$ seems suspiciously short for a cutoff of 20000 Hz; likely the transition is too abrupt, so the FIR filter is **not approximating your design** well.

3. Frequency response plot mismatch

- Your **filter frequency response comparison** shows the passband rolling off too early, even though $fc = 20000$ Hz.
- This is because `freqz` returns frequencies **up to Nyquist**, but your fc line is plotted at 20000 Hz, close to the edge. It looks like the filters are already attenuating at that frequency.
- The **IIR filter** order is very small ($N_{iir} = 6$) $\rightarrow$ smooth but not sharp transition, which is fine for Elliptic, but still not enough to reach the cutoff cleanly.

4. Spectrum of filtered signals

- In your time-domain plots, the signals look almost identical (because most frequencies pass), but in the **frequency domain**, the FIR and IIR filters show attenuation **before the actual 20000 Hz cutoff**.
- This is expected when designing high-cutoff filters with finite precision and low order, especially with the **ellipord** function which tries to meet ripple specs but keeps N minimal.

5. Summary of issues

1. Cutoff too close to Nyquist $\rightarrow$ filters behave strangely.
2. FIR filter length may be insufficient $\rightarrow$ stopband attenuation not accurate.

3. IIR filter order too small → roll-off not reaching exactly 20000 Hz.
 4. Plotting cutoff line at 20000 Hz near Nyquist exaggerates the impression that filters are “wrong”.
-

✓ How to fix

- Limit `fc` to $< 0.9 * \text{Nyquist}$ (~19800 Hz for `fs=44100`).
 - Increase FIR filter length for very high cutoff OR adjust stopband specs (`rs`) to reduce `N`.
 - For IIR, allow slightly higher order if you want sharper transition.
 - Use `np.clip` for `ws` and `fc/(fs/2)` to avoid exceeding Nyquist.
 - Plot cutoff lines slightly lower than Nyquist for clarity.
-

If you want, I can **rewrite your high-cutoff filter block** so the **frequency response plot looks correct** and both FIR/IIR filters actually pass almost everything up to ~20000 Hz without weird attenuation near Nyquist.

Do you want me to do that?